Monday 1:1 Pack — build plan

FOLIOS · FOLIOSHQ.COM

LAST UPDATED: 2026-06-19

GAME PLAN **Phase 2 #1 "SHINE"** — the first promotion-visible feature after the Pip-architecture (F1→F6) sequence. Chris runs his Monday 1:1 with his boss from Folios; this auto-builds a prep pack so he is **never flat-footed** (his #1 fear).

This is mostly **assembly + framing over data that already exists** — not new capture. The design is **LOCKED with Chris**; this doc resolves the open *how* questions and is the contract the build follows (the F-series playbook: plan-first, one concern per gated commit).

1. What it is (the locked spec)

FORMAT = hybrid: a short Pip read on top, then scannable sections. **ORDER**, top to bottom:

0. **Pip read** — 1–3 sentences framing the week. *The only Sonnet call; everything else is deterministic.*
1. **YOUR WORD** *(leads)* — promised-vs-done: commitments from the week, each **Kept / Slipped / Open**, with the account.
2. **BOSS'S OPEN ASKS, pre-answered** — "where are we on X / is Y happy" items pulled from the last 1:1, each with its **current status** auto-attached.
3. **WHAT MOVED, BY ACCOUNT** — the week's per-account delta: meetings, Gauge status pulses, deliveries.
4. **WHO HAS THE BALL** — waiting-ons: what Chris owes vs. what's owed him.

SURFACE = both:

- A **Home card Monday morning** (the approved Home design says "Monday leads with the 1:1 pack"). It teases the read + counts and opens the full pack.
- The **full pack inside the 1:1 cadence hub** (`CadenceHub` , person cadence).

THE ONE NEW MECHANIC (no new typing for Chris): boss's-asks are **extracted by Pip** from the most recent 1:1 meeting's `notes` / `pip_summary` + open leadership tasks tagged to the 1:1 cadence (`folio_tasks.cadence_id` , `account_id IS NULL`). **No tagging UI in v1** — read what he already captures.

Data Line Rule: directional only, never numbers — enforced in the prompt (both the read and the extraction).

---

2. Design decisions (the open questions, resolved)

2.1 How do we identify "the Monday 1:1 cadence"?

The cadence model already has everything (verified against `schema.sql` + `useCadences.js` + `cadenceUtils.js`):

- A 1:1 is a **person cadence**: `folio_cadences.cadence_scope = 'person'` , `contact_id` set, `account_id` usually null.
- Day-of-week convention is `Date.getDay()` (**0 = Sunday, 1 = Monday**), via the `day_of_week int check (0..6)` column. So a Monday 1:1 has `day_of_week = 1` .
- `getNextOccurrence(cadence, fromDate)` already computes the next instance for weekly/biweekly/monthly.

Resolution — two scopes, deliberately:

- The **pack SECTION renders for ANY person cadence** (it's genuinely useful prep for any 1:1, and keeps the surface coherent — App Coherence Rule). The

"boss asks" extraction + leadership tasks are naturally per-person-cadence.

- The **Home CARD auto-selects the Monday 1:1**: `pickMondayCadence(personCadences, today)` picks weekly/biweekly person cadences with `day_of_week === 1`, tie-broken by earliest `meeting_time` then `created_at`. If several Monday 1:1s exist, Home leads with that one; the rest are reachable via their own hubs.
- **Show window for the Home card:** today is Monday, OR the picked cadence's next occurrence is within ~1 day (so a Sunday-evening heads-up + Monday morning). Outside that window the card is absent (Home isn't cluttered the rest of the week); the pack is still reachable any day inside the 1:1's hub.

No hardcoding of "the boss" — it's "your Monday 1:1," derived from the cadence Chris already set up. If he has no Monday person cadence, the Home card simply never shows; the in-hub pack still works for any 1:1.

2.2 Generation strategy — build once, cache, refresh on real change

Cost is the design driver (Chris is cost-sensitive; F3 just cut Pip spend 70–90% by going event-driven). The pack splits cleanly:

- **Sections 1, 3, 4 are 100% deterministic** — computed client-side from data already loaded / cheaply queried. **Zero AI cost, always fresh.**
- **Only sections 0 (read) + 2 (boss-asks) need Pip** — and they fold into **ONE Sonnet call** (`api/monday-pack.js`) returning `{ read, boss_asks: [...] }`.

Caching (the F3 principle): the Sonnet output is cached on the cadence row and **regenerated only when a content fingerprint changes**, so a quiet week never re-bills.

- **Where:** new columns on `folio_cadences` — `pack jsonb`, `pack_fingerprint text`, `pack_generated_at timestamptz`, `pack_week date`. **Precedent:** the same table already carries `pip_brief` / `pip_brief_at` for the per-cadence brief, and operator state caches on `folio_pip_account_state`. Additive, harmless before

the code ships (F2 migration philosophy). Written via the existing `updateCadence` path (RLS = `user_id`, already user-scoped).

- **Gate:** regenerate iff `pack_week !== thisMonday` **OR** `pack_fingerprint !== currentFingerprint`. `pack_week` covers the weekly rollover; the fingerprint covers within-week changes (a new 1:1 logged, a leadership task added/closed, a commitment closed, a Gauge pulse posted).
- **Cost:** ~**1 Sonnet call per week** per Monday 1:1. Within a week: $0 unless something real moved. Model `PIP_MONDAY_PACK_MODEL || claude-sonnet-4-6` — Sonnet is the right tier (synthesis + extraction, low-frequency, promotion- visible / high-stakes, same class as the daily brief), env-overridable with no deploy (the item-48 A/B lever pattern).

Fingerprint — must be TIME-STABLE (Sanity-Pass / F3 drift lock): built ONLY from stored ids / timestamps / counts — never `Date.now()`, never "Xd ago". A relative-time leak would change the hash daily → regenerate daily → the cost cut evaporates. Inputs: last-1:1 meeting id + its `updated_at`; leadership tasks open count + done count + `max(updated_at)`; week commitments ids + done state + `max(updated_at)`; week meetings count + `max(updated_at)`; Gauge `status_updates` `max(at)`; and the Monday-week anchor. `mondayPack.test.js` asserts "+1 day over identical data ⇒ same fingerprint" (the drift lock).

Note vs. F3: F3 computes its fingerprint **server-side** to avoid a client/server divergence trap. Here the fingerprint is computed **client-side** — and that's correct, because the *gate decision is itself client-side* (the client decides whether to POST and then writes `pack` / `pack_fingerprint` / `pack_week` back to the row). Only ONE party computes it, so there is no divergence. Documented so a future reader doesn't "fix" it into the server.

2.3 Promised-vs-done (Section 1, YOUR WORD)

Commitments = `folio_tasks.is_commitment = true`. Over the **week window** (from last Monday — the previous 1:1 occurrence — to now):

- **Kept** — `done = true` AND `closed_at` within the window (a promise kept this week).
- **Slipped** — open (`!done`) AND `due_date < today` (overdue commitment).
- **Open** — open, due this week or no due date, not yet overdue.

Each row carries its account name (resolved from `account_id`). Sorted Slipped → Open → Kept (slips lead — that's the flat-footed risk). The hook queries the window directly (`folio_tasks` has commitment + open indexes), so closed-and- filtered-out commitments are still caught.

2.4 Account context & the parity rule

The task says route account context through the shared `src/lib/accountContext.js` builder. **Resolution:** the Monday pack is a **cross-portfolio weekly digest** — the same class as `api/portfolio-brief.js` and `api/leadership-readout.js` , which both **assemble compact portfolio lines** rather than dump full per-account `buildAccountContext` blocks. Using the full per-account renderer for every account here would be the wrong altitude (a week digest, not a deep single-account read) and needlessly costly.

So the pack follows the **portfolio-brief precedent**: the deterministic sections ARE the compact "current state," and they're what Pip answers boss-asks against. The parity rule's actual target is the **per-account** chat/summarize/operator drift (three renderers of one account) — not portfolio digests, which legitimately roll up. This is honest reconciliation, not a dodge: if a future version wants a *deep* read on a specific boss-asked account, it routes that one account through `renderAccountContext(account, { surface: "brief" })` . No new per-account renderer is written (the thing the rule forbids).

2.5 Sanity-Pass — the boss-ask extraction (the novel/risky part)

This is the one genuinely new mechanic; trace it end-to-end:

- **Feeds it:** the most recent **summarized** 1:1 meeting for THIS person cadence (`folio_meetings` where `cadence_id = cadence.id` , newest with `notes` or `pip_summary`), plus open leadership tasks (`folio_tasks` `cadence_id = cadence.id` , `account_id IS NULL` , `!done`). Pip extracts directional asks and states current status against the compact current-state lines passed alongside.
- **Last 1:1 has no notes/summary** (only a scheduled/draft, or notes empty) ⇒ nothing to extract ⇒ `boss_asks = []` ⇒ Section 2 renders a calm empty state ("No open asks captured from your last 1:1."). **Never a broken card.**
- **No prior 1:1 at all** (first week) ⇒ same empty state; the read still works off the deterministic sections.
- **Sonnet call fails / times out** ⇒ render Sections 1/3/4 (deterministic) + the last cached read/asks if any + a soft "couldn't refresh Pip's read" note. The pack is never blocked by the AI call.
- **Data Line:** the prompt forbids soliciting/echoing numbers, volumes, rosters — asks and statuses are directional ("trending up", "waiting on legal"), never "$X" or shop counts. Raw 1:1 notes are Chris's verbatim text (his notebook) and are only *read*, never rewritten.

---

3. Architecture (files)

File	Role
`src/lib/mondayPack.js`	**Pure** module: `pickMondayCadence()`, `buildPackSections(bundle)` (deterministic sections 1/3/4), `computePackFingerprint(bundle)`, `buildPackPromptPayload(bundle)`. No Supabase/React/fetch → unit-testable.
`src/lib/mondayPack.test.js`	Drift lock (+1-day ⇒ same fingerprint), Kept/Slipped/Open classification, empty-state shapes, `pickMondayCadence` selection.
`api/monday-pack.js`	ONE Sonnet call. In: last-1:1 notes/summary + leadership tasks + compact current-state lines + profileProse/facts. Out: `{ read, boss_asks:[{ask,status,account}] }`. JWT-scoped client + `logPipUsage` + `maxDuration:60`. Registered in `scripts/test-api-imports.js`.
`src/lib/pip.js`	`callMondayPackPip(payload)` client helper (mirrors `callPortfolioBriefPip`).
`src/hooks/useMondayPack.js`	Gathers the window data (scoped queries), computes deterministic sections + fingerprint via the pure module, reads cache off the cadence, regenerates via the endpoint when gated, writes `pack` / `pack_fingerprint` / `pack_week` back. Returns `{ sections, read, bossAsks, loading, error, generatedAt, refresh }`.
`src/views/cadence/MondayPackSection.jsx`	Renders the full pack (sections 0–4) inside `CadenceHub` for person cadences.

`src/views/home/MondayPackCard.jsx`	Home teaser card (read + counts + open-pack CTA), shown in the Monday window.
`src/App.jsx`	`onOpenPersonHub(cadence)` handler for Home (reuses the existing contact→parent-account→ `pendingPersonHubCadenceId` path).

DB: `supabase/monday_1on1_pack.sql` (the 4 `folio_cadences` columns) + fold into `schema.sql`, applied to prod via MCP migration.

---

4. Build sequence (gated commits)

Every commit gated: `vite build` · `vitest` · `node scripts/check-guards.js` · `node scripts/test-api-imports.js`. Mobile inputs ≥16px, both themes.

1. **This plan doc.**
2. **DB migration** (`folio_cadences` pack columns, MCP + `schema.sql`) + **pure `mondayPack.js` + tests** (deterministic sections, fingerprint drift lock, `pickMondayCadence`).
3. **`api/monday-pack.js`** + register in `test-api-imports` + `callMondayPackPip` in `pip.js` .
4. **`useMondayPack` hook** + **`MondayPackSection`** wired into `CadenceHub` person-cadence render.
5. **`MondayPackCard`** on Home + App `onOpenPersonHub` wiring + show-window logic.
6. **Docs** (`product-overview.md` capability + `upgrades.md` entry) + regen PDFs

- CLAUDE.md handoff.

---

5. Cost & compliance summary

- **~1 Sonnet call / week** per Monday 1:1; $0 within a week unless real change (fingerprint-gated). Deterministic sections are free and always fresh.
- **Data Line Rule** enforced in `api/monday-pack.js` (directional only; raw notes read-only). Documented in `docs/data-handling.md` / `docs/ai-governance.md` on ship if a new retention surface is introduced (it isn't — the pack cache stores Pip-authored directional prose, same class as `pip_brief`).
- **No deploy** — branch `claude/monday-1on1-pack-7cb6ia`; Chris fast-forwards `main` after testing (F-series playbook).